

架构设计与C语言面向对象设计模式

指导原则

所有模块设计必须遵循SOLID原则和OOP（面向对象）思维。关于SOLID原则的详细解释和GoF设计模式在C语言中的实现，请参阅 `02_设计模式.md`。

核心原则一览：

- **SRP（单一职责原则）**：一个模块 = 一个职责 = 一个变更理由
- **OCP（开闭原则）**：通过函数指针表扩展功能，而非修改现有代码
- **LSP（里氏替换原则）**：所有接口实现必须遵守相同的契约
- **ISP（接口隔离原则）**：拆分臃肿的接口；使用者只依赖其所需的部分
- **DIP（依赖倒置原则）**：高层模块依赖抽象，而非底层模块

C语言中的面向对象设计

虽然C语言没有类，但所有模块必须遵循OOP原则。这对于嵌入式固件的可维护性和安全性至关重要。

封装模式

每个模块对外暴露公共接口（头文件），隐藏内部实现细节：

```
/* —— uart_driver.h（公共接口） —— */
typedef struct UartDriver UartDriver_t;

typedef void (*UartErrorCb_t)(UartDriver_t *self, uint32_t err_code);
typedef void (*UartRxCb_t)(UartDriver_t *self,
                           const uint8_t *data,
                           uint16_t len);

UartDriver_t *UartDriver_Create(uint8_t port,
                               uint32_t baudrate);
void UartDriver_Destroy(UartDriver_t *self);
```

```
int  UartDriver_Open(UartDriver_t *self);
int  UartDriver_Close(UartDriver_t *self);
int  UartDriver_Send(UartDriver_t *self,
                    const uint8_t *data,
                    uint16_t len);
void UartDriver_SetErrorCb(UartDriver_t *self,
                          UartErrorCb_t cb);
void UartDriver_SetRxCb(UartDriver_t *self,
                       UartRxCb_t cb);
```

规则

1. **禁止从模块外部直接访问结构体成员。** 必须通过getter/setter函数或公共API进行访问。
2. **设备的打开/关闭必须通过模块的API进行。** 在编写设备访问代码之前，先搜索项目中其他模块是如何 打开/关闭同类设备的，并严格遵循相同的模式。
3. **不透明指针模式** - 在头文件中使用前向声明（`typedef struct Foo Foo_t;`），仅在 `.c` 文件中定义结构体。

状态通知的回调模式

当驱动或模块检测到状态变化（错误、完成、数据就绪）时，必须通过已注册的回调函数通知应用层——绝不能使用全局变量 或从外部进行轮询。

回调设计规则

1. 在公共头文件中定义回调类型（typedef）
2. 提供注册函数（`SetXxxCb`）
3. 文档标明回调触发的上下文（ISR、工作线程还是主循环）
4. 回调必须简短且非阻塞；如果应用层需要重度处理，应将任务投递到自己的队列中

示例：错误通知

```
/* 驱动实现中 */
static void handle_hw_error(MyDriver_t *self,
                          uint32_t err)
{
    self->state = DRIVER_STATE_ERROR;
    if (self->error_cb != NULL) {
        self->error_cb(self, err);
    }
}
```

```
}  
}
```

接口隔离

将大型接口拆分为聚焦的子接口：

```
/* 反面示例 - 臃肿的单体接口 */  
typedef struct {  
    int (*open)(void);  
    int (*close)(void);  
    int (*read)(uint8_t *, uint16_t);  
    int (*write)(const uint8_t *, uint16_t);  
    int (*set_baudrate)(uint32_t);  
    int (*set_parity)(uint8_t);  
    int (*get_status)(void);  
    int (*reset)(void);  
    int (*firmware_update)(const uint8_t *, uint32_t);  
} MonolithicOps_t;  
  
/* 正面示例 - 隔离的接口 */  
typedef struct {  
    int (*open)(void *ctx);  
    int (*close)(void *ctx);  
} DeviceLifecycle_t;  
  
typedef struct {  
    int (*read)(void *ctx, uint8_t *buf, uint16_t len);  
    int (*write)(void *ctx, const uint8_t *buf,  
                uint16_t len);  
} DeviceIO_t;  
  
typedef struct {  
    int (*set_baudrate)(void *ctx, uint32_t baud);  
    int (*set_parity)(void *ctx, uint8_t parity);  
} DeviceConfig_t;
```

层级边界规则 — 禁止跨层API调用

关键规则 — 历史教训：应用层曾直接调用 `HAL_GetTick()`，绕过了RTOS抽象层。

项目采用五层分层架构，每一层只依赖下一层的抽象接口：

应用层（**Actor**状态机 + 业务逻辑）
↓ 仅调用驱动层**API**和中间件**API**
中间件层（通信协议栈、错误管理、数据记录等）
↓ 调用驱动层**API**
驱动层（**OOP**设备驱动：电机、传感器、显示屏等）
↓ 仅调用**Platform**层统一接口
平台层（设备模型 + **RTOS**抽象 + 自动初始化）
↓ 仅**board**层代码调用**HAL**
HAL层（芯片厂商提供：STM32 **HAL**等）
↓
硬件寄存器

规则： 1. 应用层绝不能直接调用**HAL**函数或**Platform**函数（如 `HAL_GetTick`、`HAL_GPIO_ReadPin`） 2. 应用层只通过驱动层和中间件的公开**API**访问硬件 3. 驱动层通过 `platform_device` 统一接口操作硬件 4. 仅 `platform/arch/board` 层代码可以直接调用**HAL** 5. 每一层只依赖下一层的抽象接口，禁止跨层调用 6. 使用**RTOS**或平台抽象层替代直接**HAL**调用： - 时间： `osKernelGetTickCount()` 而非 `HAL_GetTick()` - **GPIO**：
`platform_pin_read()` 而非 `HAL_GPIO_ReadPin()` - 延时： `osDelay()` 而非 `HAL_Delay()`

Platform设备模型

这是整个分层架构的核心机制，类似Linux内核的字符设备模型。后续会有配套视频详细讲解，B站搜索「兆鸣嵌入式」观看。

设备结构体

每个硬件设备抽象为一个 `platform_device`：

```
typedef struct platform_device {  
    const char                *name;  
    uint16_t                  ref_count;  
    uint16_t                  flag;  
    os_mutex_t                dev_mutex;  
    platform_device_rx_indicate_t rx_indicate;  
    platform_device_tx_complete_t tx_complete;  
    const struct platform_device_ops *ops;  
    void                      *user_data;  
} platform_device_t;
```

设备操作集（虚函数表）

```
typedef struct platform_device_ops {
    err_t (*init)(platform_device_t *dev);
    err_t (*open)(platform_device_t *dev, uint16_t oflag);
    err_t (*close)(platform_device_t *dev);
    size_t (*read)(platform_device_t *dev, off_t pos,
                  void *buf, size_t size);
    size_t (*write)(platform_device_t *dev, off_t pos,
                  const void *buf, size_t size);
    err_t (*control)(platform_device_t *dev, int cmd,
                  void *args);
} platform_device_ops_t;
```

设备生命周期

1. **注册**: `platform_device_register(dev, "uart1", flags)` — 将设备注册到全局设备表
2. **查找**: `dev = platform_device_find("uart1")` — 按名称查找已注册设备
3. **打开**: `platform_device_open(dev, oflag)` — 自动初始化 + 引用计数+1
4. **读写**: `platform_device_read/write(dev, pos, buf, size)`
5. **控制**: `platform_device_control(dev, cmd, args)` — 配置波特率、模式切换等
6. **关闭**: `platform_device_close(dev)` — 引用计数-1, 归零时自动清理

如何写一个新的Platform设备驱动

步骤: 1. 实现 `platform_device_ops` 函数集 (init/open/close/read/write/control) 2. 在 board 层定义设备实例并注册 3. 使用 `INIT_DEVICE_EXPORT` 自动注册 (见下文"自动初始化机制")

```
/* 步骤1: 实现ops */
static err_t my_uart_init(platform_device_t *dev) { /* ... */ }
static err_t my_uart_open(platform_device_t *dev,
                          uint16_t oflag) { /* ... */ }
static size_t my_uart_read(platform_device_t *dev,
                          off_t pos, void *buf,
                          size_t size) { /* ... */ }

/* ... 其余ops ... */

static const platform_device_ops_t s_uart_ops = {
    .init    = my_uart_init,
    .open    = my_uart_open,
    .read    = my_uart_read,
    /* ... */
}
```

```
};

/* 步骤2: board层注册 */
static platform_device_t s_uart1_dev;

static void board_uart1_register(void)
{
    s_uart1_dev.ops = &s_uart_ops;
    platform_device_register(&s_uart1_dev, "uart1",
                            DEVICE_FLAG_RDWR);
}

/* 步骤3: 自动注册 */
INIT_DEVICE_EXPORT(board_uart1_register);
```

自动初始化机制

使用编译器 section 属性，让各模块的初始化函数在编译期自动收集、启动时按优先级顺序执行，无需手动编排调用顺序。后续会有配套视频详细讲解，B站搜索「兆鸣嵌入式」观看。

七个初始化级别

级别	宏	用途	示例
1	INIT_BOARD_EXPORT(fn)	板级硬件初始化	引脚配置、时钟
2	INIT_PREV_EXPORT(fn)	纯软件预初始化	平台API、堆管理
3	INIT_DEVICE_EXPORT(fn)	设备注册	UART/SPI/I2C设备注册
4	INIT_COMPONENT_EXPORT(fn)	组件初始化	ADC通道配置、传感器配置
5	INIT_ENV_EXPORT(fn)	环境初始化	文件系统挂载
6	INIT_APP_EXPORT(fn)	应用初始化	Actor启动
7	INIT_SYSTEM_READY_EXPORT(fn)	系统就绪	标记应用开始运行

使用方式

```
static void uart_board_register(void)
{
    /* 注册UART设备到platform设备框架 */
}
INIT_DEVICE_EXPORT(uart_board_register); /* 自动在Level 3执行 */

static void app_main_start(void)
{
    /* 启动应用层Actor */
}
INIT_APP_EXPORT(app_main_start); /* 自动在Level 6执行 */
```

实现原理

利用编译器 `__attribute__((section))` 将函数指针放入特定链接器 section，启动时遍历执行。各模块独立注册，无需修改主初始化文件。

```
/* 宏展开原理示意（简化） */
#define INIT_EXPORT(fn, level) \
    __attribute__((used, section(".init_fn." level))) \
    static const init_fn_t __init_##fn = fn
```

选择初始化级别的原则

- 依赖硬件引脚/时钟 --> Level 1 (BOARD)
- 纯软件基础设施 --> Level 2 (PREV)
- 注册设备驱动 --> Level 3 (DEVICE)
- 配置使用设备的组件 --> Level 4 (COMPONENT)
- 需要文件系统等环境 --> Level 5 (ENV)
- 启动业务逻辑 --> Level 6 (APP)
- 最终就绪标记 --> Level 7 (SYSTEM_READY)

container_of 与结构体嵌入继承

C语言实现"继承"的核心技巧：子类结构体将父类作为第一个成员嵌入，通过 `container_of` 宏从父类指针恢复子类指针。这是 Platform 设备 模型和 OOP 驱动设计的基础。

container_of 宏定义

```
#define container_of(ptr, type, member) \
    ((type *)((char *)(ptr) - offsetof(type, member)))
```

继承模式示例

```
/* 基类 */
typedef struct {
    const struct device_ops *ops;
    const char *name;
} device_base_t;

/* 派生类 - 将基类作为第一个成员 */
typedef struct {
    device_base_t    base;    /* "继承"基类 */
    SPI_HandleTypeDef *spi;    /* 派生类特有成员 */
    uint8_t          cs_pin;
} spi_device_t;

/* 从基类指针恢复派生类指针 */
spi_device_t *spi_dev = container_of(
    base_ptr, spi_device_t, base);
```

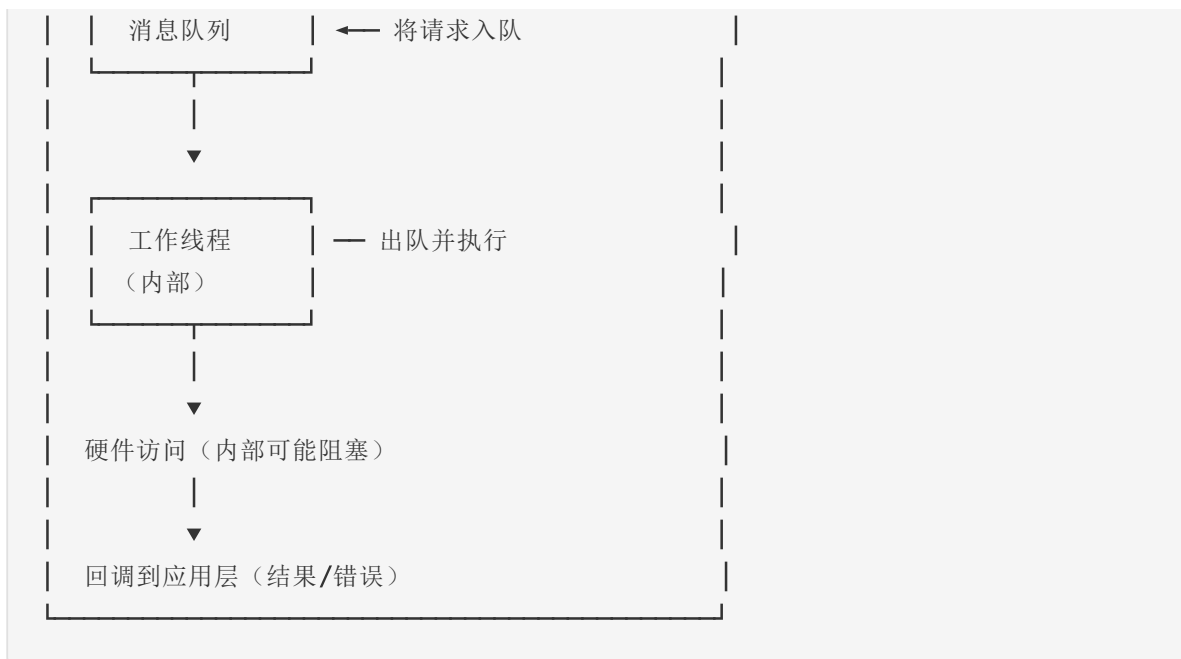
使用规则

1. 基类必须是派生类的第一个成员
2. 永远通过 `container_of` 恢复，不要强制类型转换
3. 确保 `ptr` 确实指向正确类型的成员，否则是未定义行为

驱动架构：非阻塞模式

与硬件交互的驱动必须在内部实现非阻塞。使用工作线程 + 消息队列模式：





关键点

- 公共API在入队后立即返回
- 工作线程由驱动内部创建
- 周期性轮询（如需要）在工作线程内部运行
- 结果和错误通过回调传递
- 应用层无需了解工作线程的存在
- 与业务无关的内部细节不在头文件中暴露

模块设计检查清单

在创建任何新模块之前，验证以下内容：

1. **单一职责** — 能否用一句不包含"和"字的话描述该模块？
2. **清晰的接口** — 公共API是否最小化且聚焦？
3. **依赖方向** — 是否依赖抽象而非具体实现？
4. **可测试性** — 依赖项是否可以替换以便测试？
5. **命名** — 模块名称是否清晰地传达了其用途？

驱动使用者须知

创建驱动后，务必为应用层提供以下文档：

1. **初始化顺序** - 需要先初始化什么

2. **线程安全性** - 哪些函数是线程安全的
 3. **回调上下文** - 回调在哪个线程/ISR上下文中运行
 4. **清理要求** - 正确的关闭顺序
 5. **错误处理** - 可能发生哪些错误以及如何处理
 6. **可重入性** - 哪些函数是可重入的
-

获取更多

- **B站搜索「兆鸣嵌入式」** 观看完整教程（详细讲解+代码实战）
- 公众号搜索「兆鸣嵌入式」 回复「交流」加技术交流群
- 抖音搜索「兆鸣嵌入式」看短视频精华版